MCMP117 · WEEK 5

Fundamentals of Operating Systems

Week 5 · CPU Scheduling Algorithms & Process Synchronisation

Who runs next, and how do threads share data without breaking it?

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 4 in Four Ideas

Threads

Several threads inside one process share code, data and open files; each has its own registers and stack.

CPU–I/O bursts

A process alternates CPU bursts and I/O waits. Most CPU bursts are short; a few are long.

Preemptive or not

Non-preemptive: a process keeps the CPU until it blocks or ends. Preemptive: the OS may take it away. The dispatcher does the switch.

Scoring a schedule

- **Turnaround** = completion − arrival
- **Waiting** = turnaround − burst
- **Response** = first run − arrival

Today: the algorithms that make the decision — and what happens when the threads we schedule share data.

Today's Agenda

1. **FCFS and SJF** First come first served, shortest job first, shortest remaining time first.
2. **Priority and Round Robin** Starvation and aging; time quantum; multilevel queues; what Linux does.
3. **Race conditions** Why counter++ can lose updates; the critical-section problem.
4. **Synchronisation tools** Peterson, atomic instructions, mutex locks, semaphores, producer–consumer.
5. **Hands-on** A real race in C with pthreads — and the fix, measured.

PART 1

CPU Scheduling Algorithms

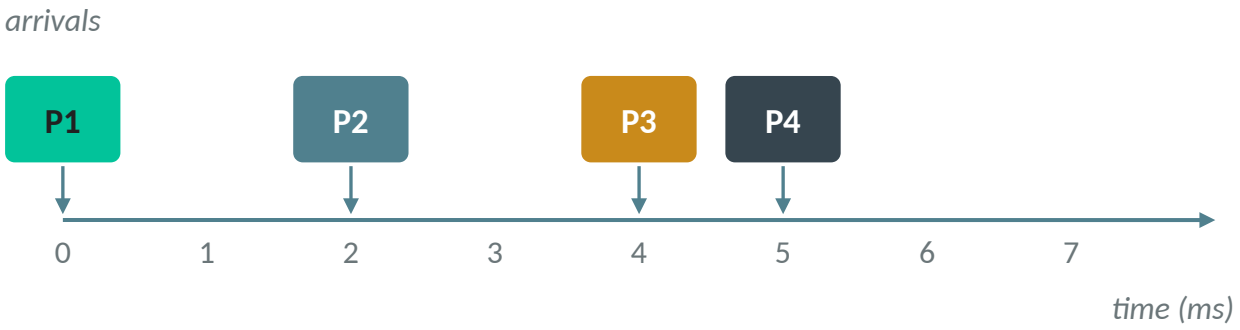
One job set, seven schedules, one comparison table.

OUR RUNNING EXAMPLE

One Job Set for Every Algorithm

Four processes. Times in milliseconds. A smaller priority number means a higher priority.

Process	Arrival time	CPU burst	Priority
P1	0	7	3
P2	2	4	1
P3	4	1	4
P4	5	4	2



Rules of the game

- **Gantt chart:** a row of boxes, one box per stretch of CPU time; ticks underneath give the times.
- **Context switch** time is taken as 0 (we count switches separately).
- **Ties** are broken by arrival time, then by name.
- **All numbers** were computed by a small simulator: code/sched_check.py.

ALGORITHM 1

First-Come, First-Served (FCFS)

The ready queue is a simple FIFO queue: whoever arrived first runs first, until it finishes or blocks. Non-preemptive.



Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	7	7	7	0	0
P2	2	4	11	9	5	5
P3	4	1	12	8	7	7
P4	5	4	16	11	7	7
Average				**8.75**	**4.75**	**4.75**

Reading the chart

- Order of arrival: P1 (0), P2 (2), P3 (4), P4 (5).
- Example, P3: turnaround 12 − 4 = 8, waiting 8 − 1 = 7 ms.
- Simple and fair in the 'queue at the bank' sense — but short jobs can wait a long time behind long ones.

The Convoy Effect: Order Matters

Three jobs arrive at t = 0 with bursts 24, 3 and 3 ms. Same jobs, same algorithm, two arrival orders.

Order A: the long job happens to be first



Order B: the short jobs happen to be first



Under FCFS the average waiting time depends entirely on luck: the order in which jobs happen to arrive.

```
Waiting times (ms)
A: P1 = 0, P2 = 24, P3 = 27
   average = 51 / 3 = 17

B: P2 = 0, P3 = 3, P1 = 6
   average =  9 / 3 = 3
```

Convoy effect: many short (I/O-bound) processes queue behind one long CPU-bound process, like cars behind a slow truck.

ALGORITHM 2

Shortest-Job-First (SJF), Non-Preemptive

When the CPU becomes free, pick the ready process with the shortest next CPU burst. Once started, a job runs to completion.



Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	7	7	7	0	0
P2	2	4	12	10	6	6
P3	4	1	8	4	3	3
P4	5	4	16	11	7	7
Average				**8**	**4**	**4**

How it was decided

- t = 0: only P1 is ready → P1 runs to 7.
- t = 7: P2 (4), P3 (1), P4 (4) are waiting → shortest is P3.
- t = 8: P2 and P4 tie at 4 → P2 arrived first.
- Average waiting drops from 4.75 (FCFS) to 4 ms.

ALGORITHM 2B

Preemptive SJF: Shortest-Remaining-Time-First

Whenever a new process arrives, compare its burst with the remaining time of the running process. Shorter wins the CPU.



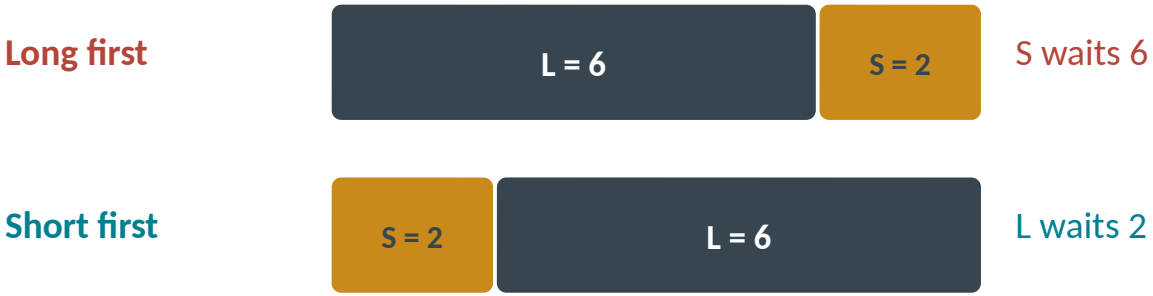
Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	7	16	16	9	0
P2	2	4	7	5	1	0
P3	4	1	5	1	0	0
P4	5	4	11	6	2	2
Average				**7**	**3**	**0.5**

Decisions at each arrival

- t = 2: P2 needs 4 < P1's remaining 5 → preempt P1.
- t = 4: P3 needs 1 < P2's remaining 2 → preempt P2.
- t = 5: P2 (2) < P4 (4) < P1 (5) → P2, then P4, then P1.
- Average waiting: 3 ms — the best of the day.

Why Shortest-First Minimises the Average Wait

An exchange argument: take any two neighbouring jobs in the schedule, a long one L before a short one S.



The argument

- Swapping the pair changes nothing for the jobs before or after it.
- Inside the pair the total wait falls from L to S.
- So any schedule with a longer job before a shorter one can be improved → sorting by burst length is optimal.

Two precise statements

- **SJF** is optimal when all jobs are ready at the same time.
- **SRTF** is optimal when jobs arrive at different times (context-switch cost ignored).

The catch: the scheduler would have to know the length of the next CPU burst — before the process runs it.

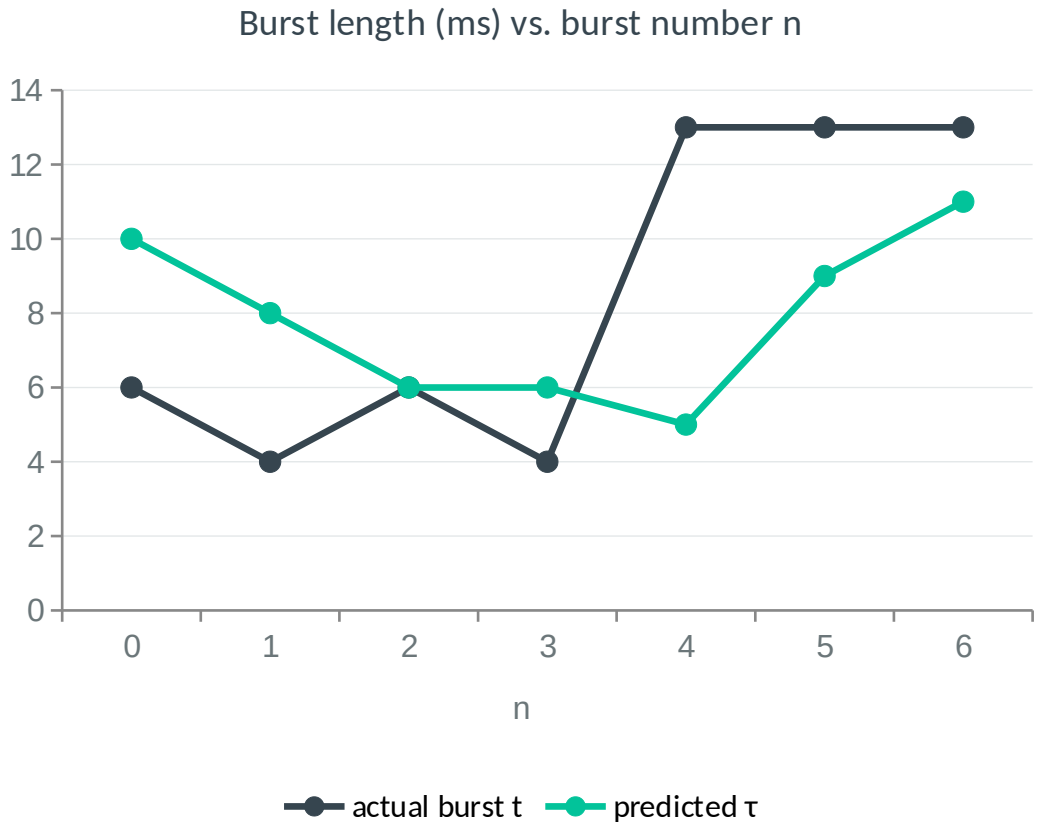
Predicting the Next CPU Burst

The OS cannot know the future, but it can guess from the past: recent bursts are good predictors of the next one.

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$$

- t_n = actual length of the n-th burst; τ_n = what we had predicted for it.
- **α** ($0 \le \alpha \le 1$) weights recent history. Common choice: $\alpha = 1/2$.

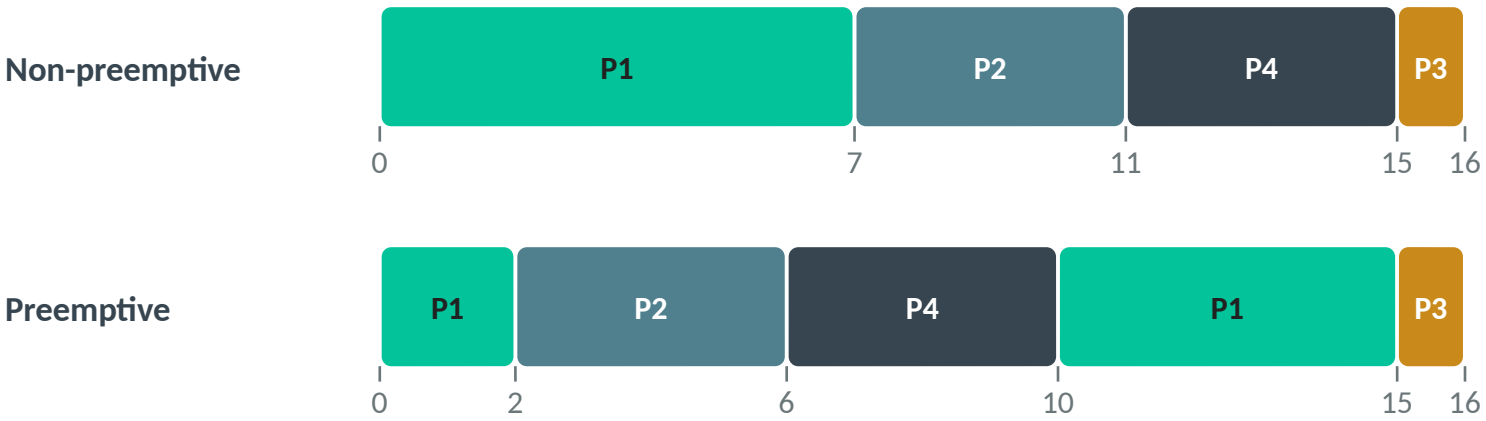
```
One step, with α = 1/2, τ0 = 10 ms, t0 = 6 ms:
  τ1 = 0.5 · 6 + 0.5 · 10 = 8 ms
  τ2 = 0.5 · 4 + 0.5 ·  8 = 6 ms     (t1 = 4)
```



ALGORITHM 3

Priority Scheduling

Each process gets a priority number; the CPU goes to the highest priority (here: smallest number). SJF is the special case priority = burst length.



Process	Priority	Wait (non-pre.)	Wait (pre.)
P1	3	0	8
P2	1	5	0
P3	4	11	11
P4	2	6	1
Average		**5.5**	**5**

What to notice

- Preemptive: P2 (priority 1) takes the CPU from P1 the moment it arrives at t = 2.
- P3 has the lowest priority (4): it waits 11 ms in both versions and runs last.
- Priorities can be internal (memory needs, burst lengths) or external (importance, who pays).

Starvation and Its Cure: Aging

In a busy system a low-priority process may wait indefinitely — it starves.

The problem: starvation

- A steady stream of higher-priority processes keeps the CPU busy.
- The low-priority process is always ready, never running.
- Folklore: when MIT shut down its IBM 7094 in 1973, it found a low-priority job submitted in 1967 that had never run.

The cure: aging

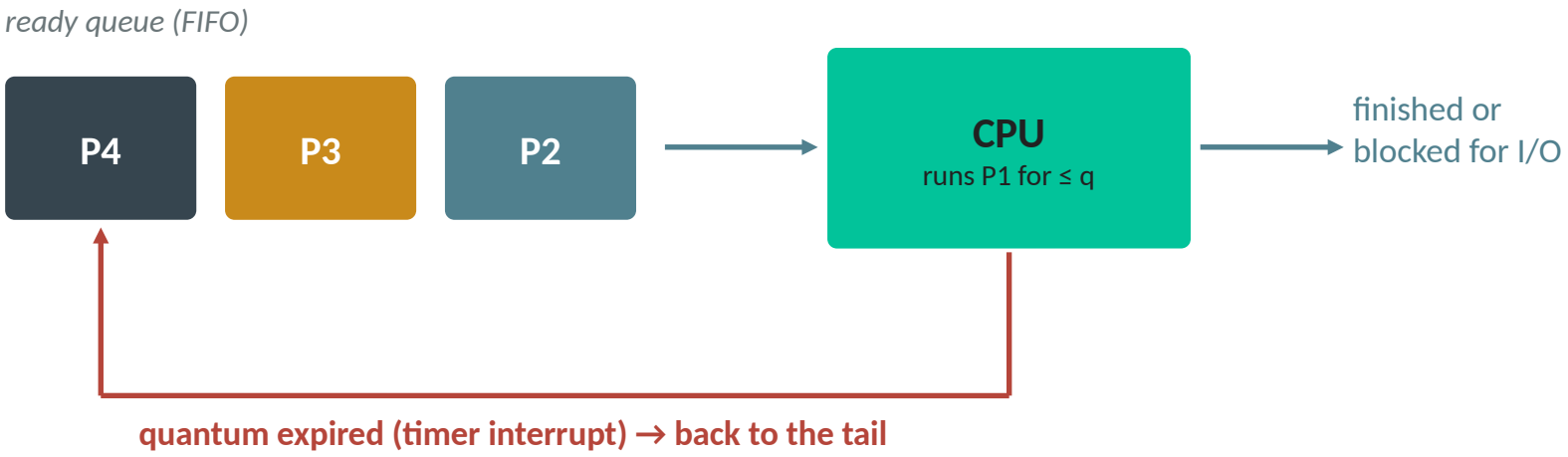
- Gradually raise the priority of a process the longer it waits.
- Example: priorities 127 (low) … 0 (high), raised by 1 every second of waiting.
- Even the lowest-priority process reaches priority 0 after 127 s = 2 min 7 s — at most.

Aging turns 'maybe never' into 'within a bounded time'. The same idea appears again as bounded waiting in Part 2.

ALGORITHM 4

Round Robin (RR): Taking Turns

FCFS plus preemption by a timer. Each process may run for at most one time quantum q, then goes to the back of the queue.



- With n processes in the queue, no process waits more than (n − 1) · q before its next turn.
- Typical quanta: 10–100 ms; a context switch takes well under 10 µs.

Our convention

- A process that arrives exactly when a quantum ends joins the queue before the preempted process.
- If a burst is shorter than q, the process leaves early and the next one starts at once.

Worked Example: Round Robin with q = 4 ms

Same four processes. Follow the ready queue after every quantum (head on the left).



Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	7	12	12	5	0
P2	2	4	8	6	2	2
P3	4	1	9	5	4	4
P4	5	4	16	11	7	7
Average				**8.5**	**4.5**	**3.25**

Ready queue

- t = 4: queue P2, P3, P1 (P2 and P3 arrived, P1 back at the tail).
- t = 8: P2 done; queue P3, P1, P4.
- t = 9: P3 done after 1 ms; P1 needs only 3 more.
- t = 12: P1 done; P4 runs alone to 16.

The Same Jobs with a Smaller Quantum: q = 2 ms

Shorter turns: everybody gets the CPU sooner, but the CPU switches far more often.



Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	7	16	16	9	0
P2	2	4	9	7	3	0
P3	4	1	7	3	2	2
P4	5	4	15	10	6	4
Average				**9**	**5**	**1.5**

Compared with q = 4

- Response time falls: average 3.25 → 1.5 ms.
- Waiting time rises: average 4.5 → 5 ms.
- Context switches: 4 → 8.
- P1 is cut into four pieces and finishes last, at t = 16.

Choosing the Time Quantum

Quantum q	Avg waiting	Avg response	Switches
1 ms	5.5	0.75	14
2 ms	5	1.5	8
4 ms	4.5	3.25	4
8 ms	4.75	4.75	3

Our job set, from sched_check.py. With q = 8 ≥ every burst, RR gives exactly the FCFS schedule.

```
Time lost to switching = s / (q + s)
with a switch cost of s = 0.1 ms:
  q =   1 ms  ->  9.1 %
  q =   4 ms  ->  2.4 %
  q =  10 ms  ->  1.0 %
  q = 100 ms  ->  0.1 %
```

q too large

RR degenerates into FCFS: interactive users wait behind long jobs.

q too small

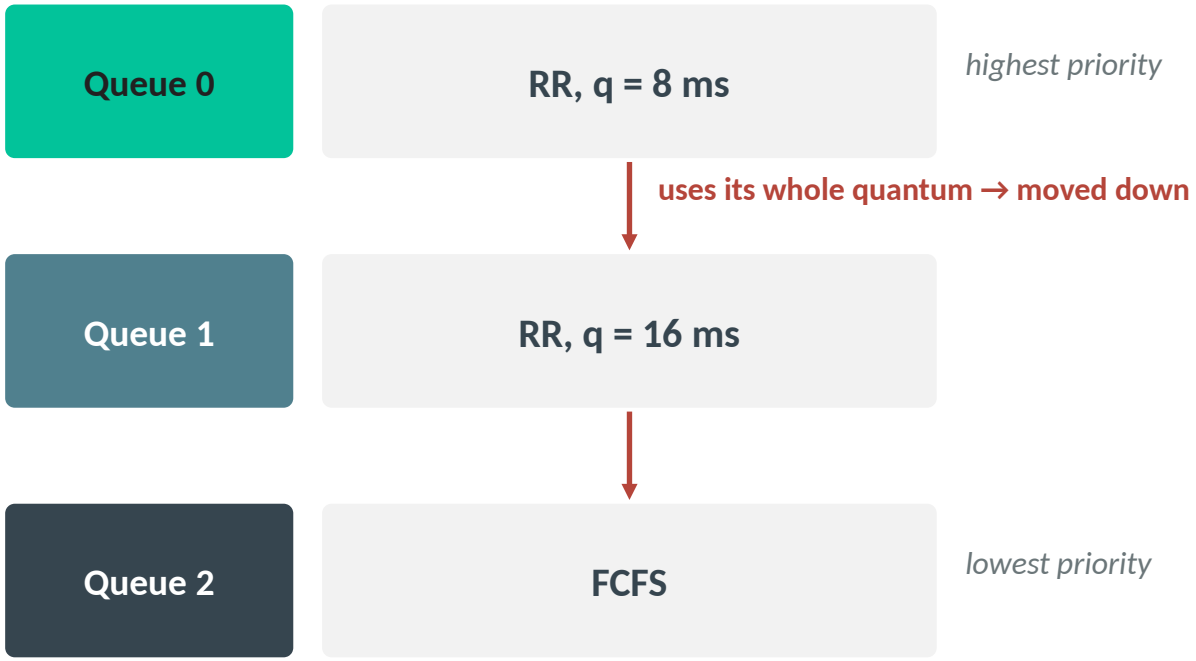
The CPU spends a large share of its time switching instead of working.

Rule of thumb: choose q so that about 80 % of CPU bursts finish within one quantum — and q must be much larger than the switch time.

OVERVIEW

Multilevel Queues and Multilevel Feedback Queues

Real systems combine algorithms: several ready queues, each with its own policy.



A new process enters Queue 0. Short, interactive jobs finish there; long CPU-bound jobs sink to the bottom.

Two variants

- **Multilevel queue:** a process is assigned to one queue for good (e.g. foreground = RR, background = FCFS).
- **Multilevel feedback queue (MLFQ):** processes move between queues according to their behaviour; waiting too long moves a process up (aging).
- The most general, and the hardest to tune.

OVERVIEW

What Linux Actually Does

Linux does not use a fixed time quantum. It tries to give every runnable task a fair share of the CPU.

Fair scheduling

- **CFS** (Completely Fair Scheduler, 2007–2023): run the task that has received the least weighted CPU time so far.
- **EEVDF** (since kernel 6.6, 2023): same fairness goal, plus virtual deadlines so that latency-sensitive tasks run sooner.
- **nice values** −20 (most CPU) … +19 (least). Each step changes the weight by about 1.25×.
- **Real-time classes** SCHED_FIFO and SCHED_RR always run before normal tasks.

```
$ taskset -c 1 ./cpu_hog &
$ taskset -c 1 nice -n 10 ./cpu_hog &
$ sleep 10
$ ps -o pid,ni,cls,pcpu,comm -C cpu_hog
  PID  NI CLS %CPU COMMAND
 7150   0  TS 88.0 cpu_hog
 7151  10  TS  9.4 cpu_hog
```

Two endless loops on ONE CPU. Weights 1024 (nice 0) vs 110 (nice 10) predict 90.3 % : 9.7 %. Measured on our 6.18 kernel: 88.0 : 9.4.

SUMMARY OF PART 1

Same Jobs, Seven Schedules

Algorithm	Preemptive?	Avg waiting (ms)	Avg turnaround (ms)	Avg response (ms)	Switches
FCFS	no	4.75	8.75	4.75	3
SJF	no	4	8	4	3
SRTF	yes	3	7	0.5	5
Priority (NP)	no	5.5	9.5	5.5	3
Priority (P)	yes	5	9	3	4
RR, q = 4	yes	4.5	8.5	3.25	4
RR, q = 2	yes	5	9	1.5	8

SRTF wins every column here — but needs to know the future. RR needs no knowledge and bounds every wait by (n − 1) · q. FCFS is simplest. Priority serves importance, not averages.

INTERMISSION

10-Minute Break

Next: what goes wrong when two threads share one variable.

PART 2

Process Synchronisation

The scheduler may switch threads at any instruction. What if they share data?

counter++ Is Not One Step

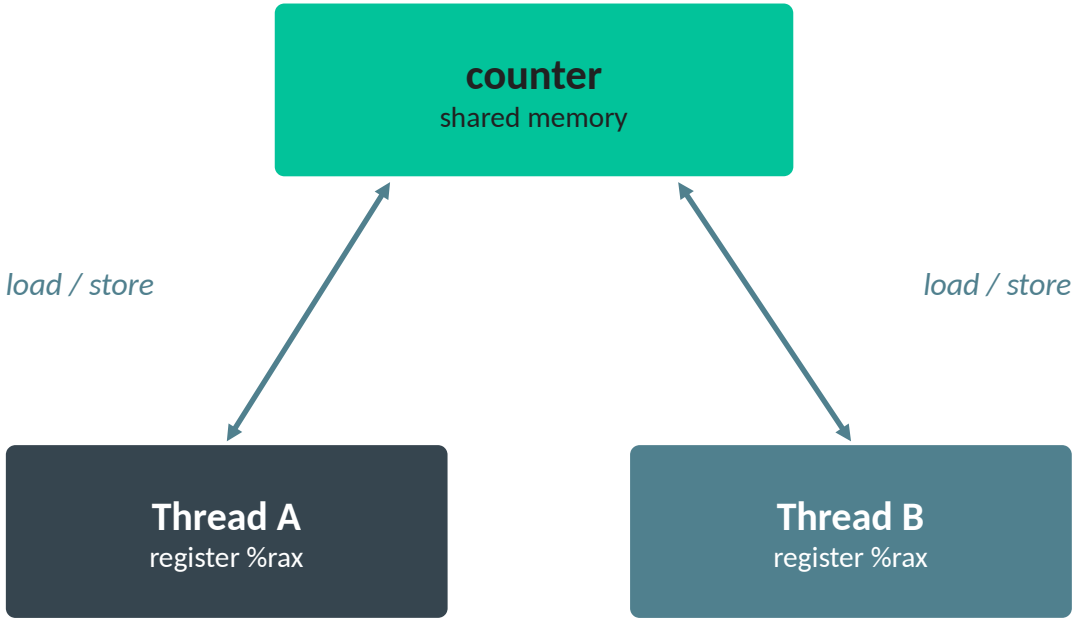
One line of C becomes three machine instructions. This is the real output of gcc -O2 -S for our race.c (x86-64).

```
counter++;          // C source

movq  counter(%rip), %rax   # 1. LOAD
addq  $1, %rax              # 2. ADD 1
movq  %rax, counter(%rip)   # 3. STORE
```

Each thread has its own registers

- counter lives in shared memory; %rax is private to each thread (saved in its thread control block on a switch).
- A switch — or another CPU — can strike between any two of the three steps.



Two threads, one variable, six instructions that can interleave in many orders.

One Unlucky Interleaving

counter starts at 5. Thread A and thread B each execute counter++ once. Correct result: 7.

Step	Thread A	Thread B	A's %rax	B's %rax	counter (memory)
1	load counter		5	–	5
2		load counter	5	5	5
3	add 1		6	5	5
4		add 1	6	6	5
5	store		6	6	6
6		store	6	6	6 ✗ (should be 7)

B's store overwrites A's: one increment is lost. Run 10 million times on two CPUs and millions are lost — we will see it live.

DEFINITION

Race Condition

Several threads access the same data concurrently, at least one writes, and the result depends on the exact order of execution.

Bank account

Two ATMs withdraw from one account at the same moment; both read the old balance, and one withdrawal disappears.

Last seat on a flight

Two booking servers both see '1 seat left' and both sell it.

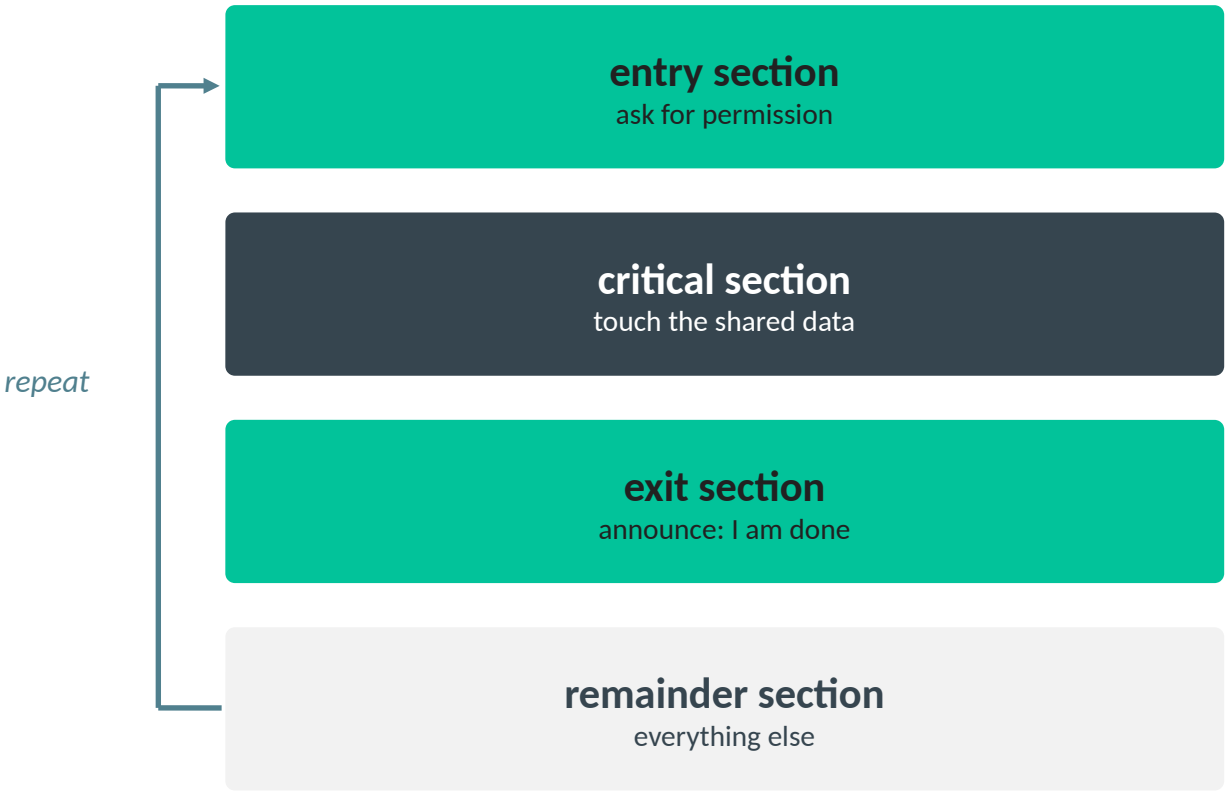
Inside the kernel

Two processes call fork() at once; without care both could receive the same new process ID.

Race conditions are timing bugs: the program can pass a thousand tests and fail on the next run. Prevent them by design, not by testing.

Critical Sections

A critical section is the code in which a thread accesses shared data. Design a protocol so that the threads never execute it at the same time.




```
while (true) {
    /* entry section     */
        counter++;   // critical section
    /* exit section      */
    /* remainder section */
}
```

The entry and exit sections are what we must design. Everything that follows today — Peterson, atomic instructions, mutexes, semaphores — is a way to build them.

Three Requirements for a Correct Solution

1. Mutual exclusion

- At most one thread is inside its critical section at any time.
- This is the safety property: the lost update becomes impossible.

2. Progress

- If the critical section is free and some threads want to enter, one of them must be allowed in soon.
- Threads in their remainder section have no veto.

3. Bounded waiting

- After a thread asks to enter, other threads can enter ahead of it only a limited number of times.
- No starvation — compare aging in Part 1.

We assume each thread runs at some non-zero speed, but make no assumption about the relative speed of the threads or the number of CPUs.

FIRST IDEA

Why Not Just Disable Interrupts?

On one CPU, a thread is switched out only by an interrupt. No interrupts during the critical section → no switch → no race.

Where it works

- A single-CPU system, inside the kernel.
- Very short critical sections (a few instructions).
- Early UNIX kernels and small embedded systems used it.

Why it does not scale

- Multiprocessors: the other CPUs keep running and can still touch the data.
- Disabling interrupts on all CPUs takes time and stalls the whole machine.
- Privileged instruction: user programs cannot do it (and must not!).
- Long critical sections delay the clock and I/O devices.

Modern systems use atomic hardware instructions instead — they work across CPUs and need no special privileges.

Peterson's Solution (Two Threads)

```
int  turn;          // whose turn is it to wait?
bool flag[2];       // flag[i]: thread i wants to enter

/* code for thread i;  j = 1 - i is the other one */
while (true) {
    flag[i] = true;          // I want to enter ...
    turn = j;                // ... but you may go first
    while (flag[j] && turn == j)
        ;                    // busy wait
    /* critical section */
    flag[i] = false;         // I am done
    /* remainder section */
}
```

Why it works

- Each thread politely gives the turn to the other.
- If both want in, turn holds only one value — the thread that wrote it last waits.
- Satisfies all three requirements (proof in the notes).
- **But:** modern CPUs and compilers reorder memory accesses, so it can fail on real hardware without memory barriers. Learn it as the idea.

Atomic Instructions: test_and_set

```
/* what the hardware does in ONE indivisible step */
bool test_and_set(bool *target) {
    bool old = *target;
    *target = true;
    return old;
}

bool lock = false;            // shared, false = free

while (true) {
    while (test_and_set(&lock))
        ;                     // spin until it was free
    /* critical section */
    lock = false;             // release
    /* remainder section */
}
```

The idea

- Read the old value and write true — as one atomic step: no other CPU can sneak in between.
- Returned false → the lock was free and is now mine.
- Returned true → someone else holds it; try again.
- Real instructions: x86 xchg and lock cmpxchg (compare-and-swap); ARM ldxr/stxr.

Mutex Locks

Programmers do not use test_and_set directly. The OS and libraries wrap it in a lock with two operations.

```
acquire(&m);        // wait until free,
                    // then take it
    /* critical section */
release(&m);        // give it back

/* POSIX threads (Linux, macOS) */
pthread_mutex_lock(&m);
    counter++;
pthread_mutex_unlock(&m);
```

Spinlock: busy waiting

- The waiting thread loops, burning CPU, until the lock is free.
- Good for very short critical sections on a multiprocessor: no context switch needed.

Sleeping lock

- The waiting thread is blocked and the CPU runs someone else; release wakes it up.
- Better for long sections. Linux pthread mutexes use futexes: no system call while the lock is free.

Semaphores

Dijkstra (1965): an integer S that is changed only by two atomic operations.

```
wait(S) {           // also called P()
    while (S <= 0)
        ;           // (or: block)
    S--;
}
signal(S) {         // also called V()
    S++;            // (and wake a waiter)
}
```

P and V come from the Dutch proberen (to test) and verhogen (to increase). In POSIX: sem_wait() and sem_post().

Binary semaphore (0 or 1)

- Start at 1: behaves like a mutex lock.
- Start at 0: forces an order — S2 in thread B runs only after thread A signals the end of S1.

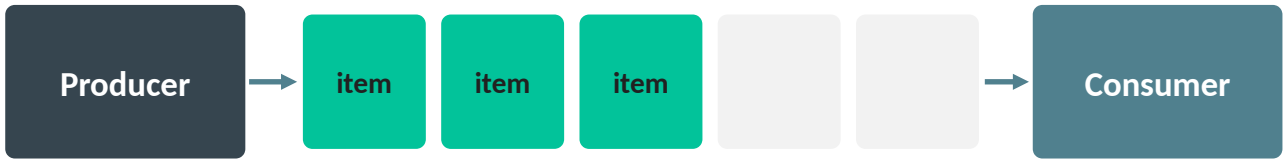
Counting semaphore (0 … N)

- Start at N = number of identical resources, e.g. 3 printers.
- Each user waits (S−−) before and signals (S++) after; the 4th user blocks until a printer is free.

CLASSIC PROBLEM

Bounded Buffer: Producer and Consumer

A producer puts items into a buffer of n slots; a consumer takes them out. Three semaphores coordinate them.



n = 5 slots: here full = 3, empty = 2

Semaphore	Initial value	Counts
`empty`	n	free slots
`full`	0	filled slots
`mutex`	1	buffer access lock

```
/* producer */
wait(empty);
wait(mutex);
 add item
signal(mutex);
signal(full);
```

```
/* consumer */
wait(full);
wait(mutex);
 remove item
signal(mutex);
signal(empty);
```

The producer blocks when empty = 0 (buffer full); the consumer blocks when full = 0 (buffer empty).

Order matters: wait on the counting semaphore first, then on mutex. Swapping them can deadlock (next slide).

A WARNING

Careless Locking Can Deadlock

Two semaphores S and Q, both initialised to 1. Each thread needs both — but takes them in a different order.

```
/* thread P0 */
wait(S);
wait(Q);
  ...
signal(S);
signal(Q);
```

```
/* thread P1 */
wait(Q);
wait(S);
  ...
signal(Q);
signal(S);
```

Time	P0	P1	S	Q
1	wait(S) ✓		0	1
2		wait(Q) ✓	0	0
3	**wait(Q) blocks**		0	0
4		**wait(S) blocks**	0	0

Deadlock: each thread waits for an event only the other can cause — both wait forever. Simplest cure: all threads acquire locks in the same global order. The same trap: a producer that takes mutex before empty while the buffer is full. (Deadlock in depth is beyond this course.)

HANDS-ON

Example 1: A Real Race in C (race.c)

```c
#define N 10000000L            /* per thread */
volatile long counter = 0;     /* SHARED     */

void *worker(void *arg)
{
    for (long i = 0; i < N; i++)
        counter++;             /* NOT atomic */
    return NULL;
}

int main(void)
{
    pthread_t a, b;
    pthread_create(&a, NULL, worker, NULL);
    pthread_create(&b, NULL, worker, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %8ld of %ld ...\n",
           counter, 2 * N);
}
/* full program: code/race.c */
```

The experiment

- Two threads, each adds 1 ten million times → should give 20,000,000.
- 28–40 % of the updates vanish — even when both threads share ONE CPU.
- Same binary, different answer on every run:

```
$ ./race          # run 4 times
counter = 12250428 of 20000000  (lost 7749572)  0.052 s
counter = 12005053 of 20000000  (lost 7994947)  0.100 s
counter = 13797565 of 20000000  (lost 6202435)  0.110 s
counter = 12046532 of 20000000  (lost 7953468)  0.080 s
$ taskset -c 0 ./race   # pinned to ONE CPU
counter = 14324099 of 20000000  (lost 5675901)  0.034 s
counter = 12867601 of 20000000  (lost 7132399)  0.034 s
```

HANDS-ON

Example 2: The Fix — and Its Price (race_fixed.c)

```
long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg)
{
    for (long i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);    /* entry */
        counter++;                    /* critical
                                         section */
        pthread_mutex_unlock(&lock);  /* exit  */
    }
    return NULL;
}
/* main() unchanged; full program:
   code/race_fixed.c */
```

Correct, but slower

- Always exactly 20,000,000.
- Unlocked (wrong): 0.05–0.11 s. Locked (right): 0.75–1.12 s — about 10–20× slower.
- Why? 20 million lock/unlock pairs, and the two CPUs fight over the lock.

```
$ ./race_fixed    # run 3 times
counter = 20000000 of 20000000  (lost       0)  1.122 s
counter = 20000000 of 20000000  (lost       0)  1.115 s
counter = 20000000 of 20000000  (lost       0)  0.749 s
```

HANDS-ON

Example 3: Producer–Consumer with POSIX Semaphores

```
sem_t empty, full, mutex;   /* SIZE, 0, 1 */

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);   /* free slot?  */
        sem_wait(&mutex);
        buffer[in] = item;  in = (in + 1) % SIZE;
        sem_post(&mutex);
        sem_post(&full);    /* one more item */
    }
    return NULL;
}
void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);    /* any item?   */
        sem_wait(&mutex);
        item = buffer[out]; out = (out + 1) % SIZE;
        sem_post(&mutex);
        sem_post(&empty);   /* one more slot */
        sum += item;   /* ... print every 5th */
    }
}
```

From pseudocode to C

- wait → sem_wait, signal → sem_post.
- sem_init(&empty, 0, SIZE): the 0 means 'shared between threads of one process'.
- Buffer of 5 slots, 20 items: the producer is blocked whenever it gets 5 items ahead.

```
$ ./prodcons
consumed  5   running sum  15
consumed 10   running sum  55
consumed 15   running sum 120
consumed 20   running sum 210
sum of 1..20 = 210 (expected 210)
```

QUICK QUIZ

Check Your Understanding

1. Jobs with bursts 5, 2, 8 ms all arrive at t = 0. Give the SJF order and the average waiting time.

2. With exponential averaging, $\alpha = 0.5$, $\tau = 8$ ms and the last burst t = 12 ms. What is the next prediction?

3. Round Robin with q = 100 ms on jobs that all need less than 50 ms behaves like which algorithm? Why?

4. Name the three requirements of a critical-section solution. Which one does 'only thread A may ever enter' violate?

Week 5 — Key Takeaways

- **FCFS** is simple but suffers from the convoy effect (17 vs. 3 ms average wait for the same jobs).
- **SJF / SRTF** minimise the average waiting time, but need the next burst length — predicted by exponential averaging.
- **Priority** scheduling can starve low-priority work; aging fixes it.
- **Round Robin** bounds every wait by (n − 1)·q; q too large → FCFS, q too small → switching overhead. Linux aims for a fair share (CFS, EEVDF since 6.6).
- **Race condition:** counter++ is load–add–store; interleavings lose updates (millions, in our test).
- **Critical sections** need mutual exclusion, progress and bounded waiting; built from atomic instructions → mutex locks and semaphores.

For the midterm (Week 8): be able to draw a Gantt chart and compute waiting, turnaround and response times for every algorithm above.

NEXT WEEK

Memory Management, Paging & Segmentation

Week 6 will cover:

- Address binding; logical vs. physical addresses; the MMU
- Swapping and contiguous memory allocation
- External and internal fragmentation
- Paging, page tables, the TLB and effective access time; segmentation

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz